

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 2

По дисциплине: «Современные методы защиты компьютерных систем»
Тема: «Защита веб-периметра и ограничение интенсивности сетевого трафика
средствами межсетевых экранов прикладного уровня»

Выполнил:
Студент 4 курса
Группы ИИ-25
Трошин И.Д.
Проверила:
Хацкевич А.С.

Цель работы: практическое освоение методов развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.

Вариант 14

Вариант	WAF: Блокируемый User-Agent	ID правила WAF	Rate Limit: Эндпоинт (Target)	Rate Limit: rate	Rate Limit: burst
14	Acunetix	10014	/api/password_reset	1r/s	1

Ход работы

```
└─kosem@lenovo ~/Desktop/CM3KC/lab2
└─$ docker compose ps
NAME                IMAGE                        COMMAND                                SERVICE    CREATED
STATUS
lab2-backend-1      lab2-backend                "python app.py"                        backend    40 seconds ago
Up 40 seconds       0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
lab2-waf-1           owasp/modsecurity-crs:nginx "/docker-entrypoint..."             waf        40 seconds ago
Up 39 seconds (health: starting) 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 8080/tcp
```

Задание 1 (Базовая фильтрация OWASP CRS):

1. Команду `curl` для проведения SQLi-атаки напрямую к бэкенду.
2. Команду `curl` для проведения SQLi-атаки через Nginx-периметр.
3. Скриншот или консольный вывод ответа Nginx, демонстрирующий блокировку атаки (403 Forbidden) базовыми правилами WAF.

1) Команду `curl` для проведения SQLi-атаки напрямую к бэкенду.

```
└─$ curl -i -G "http://127.0.0.1:5000/" --data-urlencode "id=1' OR '1'='1"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.10.21
Date: Tue, 22 Sep 2026 12:17:14 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 31
Connection: close
```

Backend OK. Endpoint reached: /

2) Команду `curl` для проведения SQLi-атаки через Nginx-периметр.

3) Скриншот или консольный вывод ответа Nginx, демонстрирующий блокировку атаки (403 Forbidden) базовыми правилами WAF.

```
└─$ curl -i -G "http://127.0.0.1/" --data-urlencode "id=1' OR '1'='1"
HTTP/1.1 403 Forbidden
Server: nginx/1.30.5
Date: Tue, 22 Sep 2026 12:19:16 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive
```

```
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>
```

При отправке SQL-инъекции через защищённый Nginx-периметр запрос был заблокирован WAF. Сервер вернул код HTTP 403 Forbidden.

Задание 2 (Пользовательские правила WAF):

1. Целевой вредоносный User-Agent и ID правила из вашего варианта.
2. Написанный вами исходный код правила (SecRule) в файле `custom.conf` для блокировки запроса.
3. Скриншот вывода команды `curl`, демонстрирующий успешную блокировку запроса с подмененным User-Agent.

1) Целевой вредоносный User-Agent и ID правила из вашего варианта.

```
└─$ curl -i -H "User-Agent: Acunetix" http://127.0.0.1/
HTTP/1.1 200 OK
Server: nginx/1.30.5
Date: Tue, 22 Sep 2026 13:26:48 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 31
Connection: keep-alive

Backend OK. Endpoint reached: /
```

2) Написанный вами исходный код правила (SecRule) в файле `custom.conf` для блокировки запроса.

```
SecRule REQUEST_HEADERS:User-Agent "@contains Acunetix"
"id:10014,phase:1,deny,status:403,msg:'Blocked Acunetix Scanner'"
```

3) Скриншот вывода команды `curl`, демонстрирующий успешную блокировку запроса с подмененным User-Agent.

```
└─$ curl -i -H "User-Agent: Acunetix" http://127.0.0.1/
HTTP/1.1 403 Forbidden
Server: nginx/1.30.5
Date: Tue, 22 Sep 2026 13:29:23 GMT
Content-Type: text/html
Content-Length: 153
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.30.5</center>
</body>
</html>
```

Задание 3 (Rate Limiting и L7 DDoS):

1. Заданные параметры `rate` и `burst` для целевого эндпоинта из варианта.
2. Фрагмент конфигурации из файла `user_nginx.conf`, содержащий добавленные директивы `limit_req_zone` и `limit_req`.
3. Текстовый вывод результатов стресс-теста утилиты `ab` (или скриншот терминала), подтверждающий наличие ответов `503 Service Unavailable` (показатель `Failed requests` или `Non-2xx responses`).

1) Заданные параметры `rate` и `burst` для целевого эндпоинта из варианта.

Rate Limit: Эндпоинт (Target)	Rate Limit: rate	Rate Limit: burst
/api/password_reset	1r/s	1

2) Фрагмент конфигурации из файла user_nginx.conf, содержащий добавленные директивы limit_req_zone и limit_req.

```
limit_req_zone $binary_remote_addr zone=mylimit:10m rate=1r/s;

server {
    listen 80;
    server_name localhost;

    location / {
        # Применение RATE LIMITING (Задание 3)
        # Раскомментируйте и настройте директиву limit_req ниже:
        #limit_req zone=mylimit burst=1 nodelay;
        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location = /api/password_reset {
        limit_req zone=mylimit burst=1 nodelay;

        proxy_pass ${BACKEND};
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

3) Текстовый вывод результатов стресс-теста утилиты ab (или скриншот терминала), подтверждающий наличие ответов 503 Service Unavailable (показатель Failed requests или Non-2xx responses).

До настройки конфигурации

```
└─$ ab -n 500 -c 10
http://127.0.0.1/api/password_reset
This is ApacheBench, Version 2.3 <$Revision:
1934973 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd,
http://www.zeustech.net/
Licensed to The Apache Software Foundation,
http://www.apache.org/

Benchmarking 127.0.0.1 (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Finished 500 requests

Server Software:      nginx/1.30.5
Server Hostname:      127.0.0.1
Server Port:          80

Document Path:        /api/password_reset
Document Length:       49 bytes

Concurrency Level:     10
Time taken for tests:   1.646 seconds
Complete requests:      500
Failed requests:         0
Total transferred:      103000 bytes
HTML transferred:       24500 bytes
Requests per second:    303.70 [#/sec] (mean)
Time per request:       32.928 [ms] (mean)
Time per request:       3.293 [ms] (mean, across
all concurrent requests)
Transfer rate:          61.09 [Kbytes/sec]
received

Connection Times (ms)
      min      mean[+/-sd] median   max
Connect:    0       0   0.2      0      2
Processing:  14      32   7.3     31     72
Waiting:    13      31   7.3     30     71
Total:      14      32   7.4     31     72

Percentage of the requests served within a
certain time (ms)
 50%      31
 66%      34
 75%      35
 80%      38
 90%      42
 95%      45
 98%      50
 99%      56
100%     72 (longest request)

└─$ ab -n 500 -c 10
http://127.0.0.1/api/password_reset
```

После

```
This is ApacheBench, Version 2.3 <$Revision:
1934973 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd,
http://www.zeustech.net/
Licensed to The Apache Software Foundation,
http://www.apache.org/

Benchmarking 127.0.0.1 (be patient)
Completed 100 requests
Completed 200 requests
Completed 300 requests
Completed 400 requests
Completed 500 requests
Finished 500 requests

Server Software:      nginx/1.30.5
Server Hostname:      127.0.0.1
Server Port:          80

Document Path:        /api/password_reset
Document Length:       49 bytes

Concurrency Level:     10
Time taken for tests:   0.525 seconds
Complete requests:      500
Failed requests:         498
      (Connect: 0, Receive: 0, Length: 498,
Exceptions: 0)
Non-2xx responses:      498
Total transferred:      184174 bytes
HTML transferred:       98204 bytes
Requests per second:    952.72 [#/sec] (mean)
Time per request:       10.496 [ms] (mean)
Time per request:       1.050 [ms] (mean, across
all concurrent requests)
Transfer rate:          342.71 [Kbytes/sec]
received

Connection Times (ms)
      min      mean[+/-sd] median   max
Connect:    0       0   0.1      0      1
Processing:  5      10   3.5      9     31
Waiting:    5      10   3.5      8     31
Total:      6      10   3.5      9     32

Percentage of the requests served within a
certain time (ms)
 50%       9
 66%      10
 75%      12
 80%      14
 90%      15
 95%      17
 98%      19
 99%      20
100%     32 (longest request)
```

Задание 4 (Проектирование профилей защиты):

1. Развернутый ответ на пункт 1: конфигурационные параметры Nginx для защиты от Slowloris (client_body_timeout, client_header_timeout, keepalive_timeout) с техническим обоснованием выбранных значений в секундах.

Атака **Slowloris** заключается в установлении большого количества HTTP-соединений с веб-сервером и намеренно медленной отправке HTTP-запросов. Злоумышленник может передавать заголовки или тело запроса небольшими фрагментами с большими интервалами между ними. Сервер считает соединения активными и продолжает выделять под них ресурсы. При большом количестве таких соединений доступные ресурсы сервера могут быть исчерпаны, из-за чего легитимные пользователи не смогут получить доступ к приложению.

Для снижения эффективности Slowloris в Nginx используются параметры, ограничивающие время ожидания данных от клиента:

```
client_header_timeout 10s;  
client_body_timeout 10s;  
keepalive_timeout 15s;
```

Параметр	Значение	Назначение
client_header_timeout	10 с	Ограничение времени ожидания HTTP-заголовков
client_body_timeout	10 с	Ограничение интервала ожидания данных тела запроса
keepalive_timeout	15 с	Ограничение времени сохранения неактивного keep-alive-соединения

Выбор 10 секунд для заголовков и тела запроса позволяет достаточно быстро закрывать соединения, которые слишком долго находятся в состоянии ожидания, что снижает эффективность Slowloris. Значение 15 секунд для keep-alive оставляет небольшой интервал для повторного использования соединения обычными клиентами, но не позволяет длительное время удерживать неактивные соединения.

2. Развернутый ответ на пункт 2: академическое описание и схема обработки трафика в эшелонированной защите Пользователь -> CDN (L3/L4 защита) -> WAF (L7 защита) -> Сервер приложения.

Эшелонированная защита представляет собой организацию информационной безопасности на основе нескольких последовательно расположенных уровней защиты. Каждый уровень выполняет собственные функции фильтрации и противодействия определённым типам угроз. Такой подход позволяет не полагаться на один механизм безопасности: если вредоносный трафик не был обнаружен на одном уровне, он дополнительно проверяется на следующем.

Первым защитным уровнем является **CDN с механизмами защиты L3/L4**. Уровни L3 и L4 соответствуют сетевому и транспортному уровням модели OSI. На данном этапе

анализируются преимущественно IP-пакеты и транспортные соединения, а не содержимое HTTP-запросов.

L3/L4-защита предназначена прежде всего для противодействия сетевым атакам. Она может выполнять фильтрацию IP-адресов, ограничивать количество соединений, обнаруживать аномальный TCP/UDP-трафик и противодействовать DDoS-атакам. CDN также позволяет распределять входящий трафик между различными узлами инфраструктуры, снижая нагрузку на непосредственно защищаемый сервер.

Например, если злоумышленник направляет на приложение огромное количество сетевых пакетов или пытается установить большое количество TCP-соединений, такой трафик может быть отфильтрован ещё до того, как достигнет WAF и сервера приложения.

После прохождения сетевого уровня разрешённый трафик поступает на **WAF (Web Application Firewall)**. WAF относится к уровню **L7**, поскольку работает на прикладном уровне и способен анализировать структуру и содержимое HTTP/HTTPS-запросов.

В отличие от L3/L4-защиты, WAF может анализировать URL, HTTP-метод, параметры запроса, HTTP-заголовки, cookies и другие элементы прикладного протокола. Поэтому WAF предназначен для обнаружения угроз, которые невозможно эффективно определить только по IP-пакетам или TCP-соединениям.

К таким угрозам относятся, например, **SQL Injection, XSS, попытки эксплуатации веб-уязвимостей, автоматизированное сканирование и подозрительные HTTP-запросы**. WAF также может реализовывать ограничения интенсивности запросов (**Rate Limiting**), позволяя ограничить количество обращений к определённому endpoint за единицу времени.

Вывод: освоил методы развертывания межсетевых экранов прикладного уровня (WAF), написания пользовательских правил фильтрации L7-угроз, а также проектирование механизмов защиты от DDoS-атак и ограничения частоты запросов (Rate Limiting) на базе обратного прокси-сервера Nginx.